

Specification V2 — Professional Interior Design Room Planner

Status: Implementation-ready contract (frozen, Rev 4)

Date: 2026-10-01

This document replaces V1. Rev 4 changes: stable vertex IDs replace array-index wall topology (§3, A1), no-deadlock rule for wall-edit-invalidated objects (A3), multi-room shared-wall boundary note (§14). Rev 3 changes: **apply** failure semantics (§9), mandatory inverse data for undo (§9), **snapDistance** default (§5), EditWall pipeline exception (A3), and a pointer to the new binding **Appendix B — UX Interaction Defaults** (separate document).

1. Product Definition & Scope

Product A standalone spatial-design tool. It is not construction-grade CAD. Designers create and edit room geometry, place doors/windows, place and manipulate furniture, receive immediate spatial feedback, work precisely in 2D, explore in 3D, and produce professional outputs.

Core architectural principle One authoritative domain model. 2D (Konva) and 3D (React Three Fiber) are independent renderers of that model. Neither renderer owns design state.

Scope categories

Category	Meaning
Core Prototype (P0)	Must work exceptionally well. Subject to early designer validation.
Architectural Foundations	Supported by the domain model. UI may be minimal or absent in the first prototype.
Deferred	Explicitly out of the first validated prototype.

P0 — Core Prototype Rectangular + irregular rooms, wall editing with per-segment thickness, doors/windows, furniture library, ghost placement, snapping, collision, containment, door/window constraints, live dimensions, Measure tool (including room area display), 2D editing (including dashed representation for objects above cut-plane), 3D visualization + basic direct manipulation, basic multi-selection (group move / rotate / duplicate / delete), inspector, undo/redo, save/load, materials, furniture metadata (vendor / sku / finish / cost), clearance visualisation toggle, saved camera viewpoints, PDF + image export.

Architectural Foundations (model support required, sophisticated UI not required initially) Multiple rooms, room height / ceiling plane, furniture assemblies, layers/visibility groups, clearance rules (hard/soft), full anchorType system (WALL / CEILING), GLTF asset path, project revisions, alternatives, cloud-ready serialization.

Deferred Sophisticated DXF, manufacturer catalogues, collaboration, multi-level buildings, advanced group scaling, extensive annotation systems, complex CAD topology tools, rooms with holes (inner rings).

2. Architectural Principles (Governing Rules)

1. No screen coordinates, viewport dimensions, or device pixel ratio may enter the domain model or persistent design state.
2. Neither Konva nor Three.js owns authoritative design state.

3. 2D and 3D use the identical spatial engine (coordinates, snapping, containment, collision, constraints, quantization).
4. Transient invalid states may exist during interaction and cannot themselves be committed. A previously committed object may subsequently become invalid as a consequence of another committed domain change (for example, room geometry editing). Such objects remain committed; their invalid status is derived by re-validation after the change and is not stored as a persistent flag on the instance.
5. Invalid drops animate silently back to the last committed valid state. No history entry is created.
6. A continuous drag produces exactly one domain command and one history entry. Multi-object operations (group move, group rotate, group duplicate, group delete) are expressed as a single composite command.
7. All committed spatial values are quantized. Linear values (x, y, elevation, width, length, height) quantize to 1 mm. Rotation quantizes to the nearest 0.1° (or to the active angular snap increment if one is set).
8. Room polygons are normalized to counter-clockwise (CCW). Wall normals are derived explicitly. Simple polygons only; holes (inner rings) are out of scope.
9. Furniture collision = 2D footprint intersection **and** vertical interval overlap.
10. Wall-attached fixtures are attachments, never free furniture.
11. UI / interaction state is strictly separated from domain state.
12. No important interaction depends on hover.
13. All touch targets $\geq 44 \times 44$ px.
14. 3D wall openings use deterministic segmented geometry, never boolean subtraction.
15. Procedural 3D furniture is replaceable via the FurnitureDefinition asset provider without changing the domain model.
16. Floating-point tolerance (computational) and product quantization are distinct concepts.
17. Boundary semantics: footprint entirely inside or exactly touching the room boundary is valid unless a clearance constraint requires otherwise. Crossing or outside is invalid.
18. Domain commands operate on domain state and produce deterministic state transitions. Commands are immutable history records and must not contain renderer-specific or transient interaction state.
`apply(command, state)` is a pure function; all new IDs are generated outside `apply` and carried inside the command.
19. **apply** is total over well-formed commands. Rejection of invalid operations (hard constraint failure, locked targets, invalid polygons) happens in the commit pipeline **before** a command is emitted. `apply` never re-validates design constraints and never throws on a well-formed command; it may throw only on malformed input (missing ID, unknown type, structurally corrupt command), with a stable `ApplyErrorCode`.
20. **Undo uses inverse-application.** Every command carries the mandatory data required to construct its inverse (see Phase 1 §4). Undo applies the inverse command; redo re-applies the original. Replay of any stored command through `apply` must never throw.

Implementation Discretion This specification defines product behaviour, domain invariants, architectural boundaries, and PO acceptance criteria. It does not prescribe internal implementation beyond those constraints.

The developer may choose implementation techniques, algorithms, module boundaries, data structures, and supporting libraries provided that:

1. all frozen domain invariants remain true;
2. the public/domain contracts remain compatible;
3. the shared spatial engine remains renderer-independent;
4. performance and testing requirements are met; and

5. any proposed change that would affect a frozen architectural principle or user-visible behaviour is raised before implementation.

Detailed edge-case and transient-state defaults are defined in **Appendix A** (this document) and UX interaction defaults in **Appendix B** (separate document). Both are binding for P0 implementation.

3. Domain Model

Project

```
Project
├─ schemaVersion // currently 1
├─ id, metadata, units
├─ shared FurnitureDefinitions[]
├─ shared Materials[]
├─ rooms[]
├─ layers[] // visibility / lock groups
├─ savedViews[] // camera viewpoints
├─ versions[] / alternatives[] // Architectural Foundation
└─ history // stores commands (serialisable form)
```

Room

```
Room
├─ id, name
├─ floorElevation // default 0
├─ wallHeight / ceilingHeight
├─ vertices (CCW, each { id, position }, in metres) // interior finished face,
simple polygon
├─ walls[] (segments referencing stable vertex IDs, per-segment thickness)
├─ fixtures[] // doors, windows – wall attachments
└─ furnitureInstances[]
```

Wall segment

```
WallSegment
├─ id
├─ startVertexId, endVertexId // stable vertex IDs – never array indices
├─ thickness // per-segment, default 150 mm, expands outward only
└─ (derived normal, length, etc.)
```

FurnitureDefinition (data-driven)

```
FurnitureDefinition
├─ id, name, category
├─ defaultWidth, defaultLength, defaultHeight
└─ footprint definition
```

```
├─ collisionPolicy
├─ clearancePolicies[] // minimum viable shape required in Phase 1
├─ materials
├─ renderer2D
└─ assetProvider3D // Procedural | GLTF
```

ClearancePolicy (minimum viable)

```
ClearancePolicy
├─ side: 'front' | 'back' | 'left' | 'right' | 'all'
├─ offset: Metres
└─ severity: 'hard' | 'soft'
```

Clearance zone geometry (binding):

- A zone is a rectangle extending **offset** metres outward from the corresponding footprint edge in the object's local (rotated) frame; **'all'** extends all four edges.
- **Room-boundary exemption:** overlap between a clearance zone and the room boundary (or the wall beyond it) is **not** a violation. Zones are evaluated against other furniture footprints, door swing sectors, and fixture access zones only. This prevents every wall-adjacent sofa from being invalid.
- **Clearance-vs-clearance overlap is not a violation.** Only object footprints are tested against zones.

FurnitureInstance

```
FurnitureInstance
├─ id, definitionId, roomId
├─ position (x, y), elevation, rotation
├─ width, length, height
├─ finishOverrides
├─ locked
├─ metadata? {
│ vendor?: string
│ sku?: string
│ finishCode?: string
│ unitCost?: number
│ notes?: string
│ }
└─ parentAssemblyId? // Architectural Foundation
```

Fixture (door / window)

```
Fixture
├─ id, type, wallId
├─ offsetAlongWall // measured along the interior wall boundary
├─ width, height, elevation
├─ orientation / hingeSide / swingAngle / accessZone
└─ type-specific parameters
```

SavedView

```
SavedView
├─ id, name
├─ cameraPosition, target
├─ projection // 'perspective' | 'orthographic'
└─ roomId? // optional association
```

Initial supported door subset Single-leaf hinged door only: hinge side, opening direction, configurable swing angle, access zone. Double, pocket, barn, bi-fold, slider are later extensions.

Door-swing vs. other doors: intersection of one door's swing sector or access zone with **another door's** swing sector/access zone is a **soft** violation in P0 (visual warning only; it never blocks commit).

Assemblies — Supported in the model (parent transform + child instances). First prototype exposes only multi-selection group operations.

Layers (initial set) — Architecture, Furniture, Fixtures, Annotations, Measurements. Each supports visible / hidden / locked.

Units Internal: metres. Display: metric or imperial. Default snap: 0.1 m / 6 in. Grid configurable. Display units independent of internal units. Imperial display uses the fractional format `3' - 4½"` (nearest ½ inch) in dimensions and inspector fields; parsing accepts the same format. Imperial display is P1; P0 may render metric only.

Quantization

- Linear values (position, elevation, width, length, height) → 1 mm.
- Rotation → nearest 0.1° (or active angular snap increment).
- Rounding mode: `Math.round` semantics on the millimetre (or 0.1°) value. Midpoint 0.0005 m rounds to 0.001 m; negatives follow the same magnitude rule.
- Rotation is stored in **radians** after quantizing the angle in degrees. Because π is irrational in binary, serialization round-trip tests for rotation must assert tolerance ($\leq 1e-9$ rad), not exact equality.
- Floating-point calculations used during pointer movement.

Floating-point policy `EPSILON = 1e-6 m` for geometric predicates. Tests use tolerance-aware comparisons. Touching boundary $\neq$ collision unless clearance requires it.

2D cut-plane convention Default cut-plane height = 1.2 m. Any furniture whose `elevation + height > cutPlaneHeight` is rendered with dashed lines in 2D. Pure rendering concern.

Validity Validity is derived by re-running the validation engine after every committed change. No persistent `isInvalid` flag is stored on instances. UI "flagged invalid" state is a projection of the latest validation result.

Locked semantics Locked objects still participate in collision and containment. They reject move, resize, rotate and delete commands. They remain selectable.

4. Spatial / Geometry Engine

Framework-independent. Pure TypeScript. Fully unit-testable.

Coordinate system Room-local metres.

- 2D: World X → Canvas X, World Y → Canvas Y (Y-down isolation lives only in the 2D adapter).
- 3D: World X → Three X, World Y → Three Z, Elevation → Three Y.

Required pure helpers: `worldToCanvas`, `canvasToWorld`, `worldToThree`, `threeToWorld`.

Room geometry

- Rectangles and irregular / concave simple polygons (no holes).
- CCW normalization + validation (reject self-intersection, zero area, degenerate walls, duplicate vertices) with stable error codes.
- Containment: triangulate validated CCW polygon (or equivalent robust method). Test furniture footprint against triangles using the boundary policy in Rule 17.
- Room polygon = interior finished face. Wall centreline is derived visual/reference geometry only.
- Each wall segment carries its own thickness (expands outward only).

Collision Convex footprints → SAT (or equivalent). Vertical: intervals overlap. Room containment handled separately from object-object collision.

Constraints Structured result:

```
{ valid, violations[], involvedObjectIds[], involvedWallIds[],  
  involvedFixtureIds[] }
```

Severity: hard (blocks commit) | soft (visual only). Priority order for feedback: outside room → door swing → fixture access → physical collision → clearance.

Door swing geometry (minimum contract)

- Hinge point derived from wall segment + `offsetAlongWall` + `hingeSide`.
- Swing sector represented as a polygonal approximation of the arc (or exact sector if the collision library supports it).
- `accessZone` is an additional rectangular or sector region beyond the leaf.
- Non-90° swing angles are supported and must be tested.

Fixture vertical validity (binding) A committed fixture must satisfy `elevation >= 0` and `elevation + height <= wallHeight` of its room (doors additionally require `elevation == 0`).

Violation → hard, type `fixture_out_of_wall`.

Snapping

```
SnapCandidate { targetType, position, distance, priority, reason }
```

Resolver ranks by distance threshold → contextual priority → user-enabled types. Default `snapDistance` = **0.25 m** (configurable interaction preference), analogous to `fixtureWallSnapDistance` = 0.5 m. Default priority order (higher number = higher priority): wall endpoint (100), wall (80), furniture edge (60), furniture centre (50), alignment (40), grid (20). Tie-break: smaller distance, then stable type order.

Clearance Defined on FurnitureDefinition via ClearancePolicy. Evaluated by the shared engine under the zone geometry rules in §3. Soft violations never block commit. Clearance zones can be visualised via a toggle (renderer concern).

5. Interaction Model

Persistent tools — SELECT, PAN, WALL_EDIT, MEASURE.

Temporary states — DRAGGING, RESIZING, ROTATING, PLACEMENT_GHOST, MARQUEE_SELECT, MOVING_VERTEX, INSERTING_VERTEX.

Ghost placement — Library selection → translucent ghost follows pointer. Same snap / containment / collision / constraint rules as real furniture. Valid = normal translucent; invalid = red. Click commits, Escape cancels.

Live dimensions (P0) — While dragging: show distances to nearest walls, nearby furniture, fixtures, and alignment axes. Snap indicators appear on successful snap. Budget and decluttering rules: see Appendix B, B10.

Measure tool (P0) — Non-destructive. Point-to-point, wall-to-wall, object-to-object, distance-to-wall. Room area is calculated and displayed (P0).

Invalid handling — During drag: red feedback + constraint message. On release: if invalid → silent animate-back to last valid state, no history entry.

Selection — Shared across 2D/3D by domain ID. P0 multi-selection supports: group move, group rotate, duplicate, delete (each as a single composite command). Group-rotate pivot = bounding-box centre of the selection (binding; see B2).

Keyboard — Arrow = small move, Shift+Arrow = larger move (respects snap), R = +45° (applied to the quantized stored rotation, not accumulated float), Delete, Escape, Ctrl/Cmd+D = duplicate.

Touch — No hover dependence. Tap select, one-finger drag move/pan, pinch zoom, long-press context, 44×44 targets.

View switching — Cancel any active drag, restore last committed transform, preserve selection, switch renderer, restore previous camera/view state for that mode.

Clearance visualisation — A toggle shows hard/soft clearance zones for the selected object(s) as dashed or hatched overlays in both 2D and 3D.

Mode escape hatches — Escape semantics per tool and state are binding defaults defined in Appendix B, B7.

6. UI Shell & UX

Interaction defaults for wall editing, rotation, resize, selection, invalid-state communication, onboarding, and history visibility are **binding** and defined in **Appendix B** (separate document). Layout, styling, and branding remain design territory and are not specified there.

Layout (desktop) — Top toolbar (56–64 px) · Left sidebar 280 px · Viewport · Right inspector 320 px · Status bar ~28 px. Both sidebars collapsible.

Responsive — Desktop landscape: three-panel. Tablet landscape: one sidebar may auto-collapse. Tablet portrait: viewport + bottom/contextual sheet (status bar roles migrate to the sheet).

Focus Mode — Collapses sidebars (and optionally toolbar).

Library — Search, category filter, recently used, favourites. Definitions remain generic/data-driven.

Inspector — Collapsible groups: Transform, Appearance, Constraints, Metadata. Single numerical source of truth. "Mixed" for multi-selection conflicts. Multi-selection numeric editing applies to any property with a defined per-instance semantic (whitelist in Appendix B, B4); others display as unavailable. Metadata fields (vendor, sku, finishCode, unitCost, notes) are editable.

HUD — Tiny on-canvas strip: Rotate / Duplicate / Lock / Delete. Never competes with the inspector.

Status bar — Three roles: Context (selection + exact position), Snap state, Constraint message (clickable → selects conflicting object). Never modal.

Room Navigator — Active room is explicit. Furniture belongs to a room. Definitions and materials are project-level.

Camera / view state — Independent of design state and history. 2D pan/zoom and 3D camera are preserved across view switches. Saved viewpoints restore with smooth transitions.

7. 2D Renderer

Konva layers (no authoritative state): Grid · Room · Fixtures · Furniture · Selection · Dimensions · Interaction Overlay · Constraint Feedback · Clearance overlays.

Blueprint-style 2D furniture (recognizable line drawings, not raw rectangles). Objects whose elevation + height exceeds the cut-plane (default 1.2 m) are rendered with dashed lines. Hit-testing for dashed (above cut-plane) objects follows B6.

8. 3D Renderer

React Three Fiber. Perspective + orthographic, orbit/pan/zoom, isometric preset (~[10,10,10]), lighting, shadows, floor grid, segmented walls (respecting per-segment thickness), procedural furniture (replaceable via asset provider).

3D manipulation path — Pointer → Raycaster → floor plane Y=0 → world point → domain coordinates → snap → validate → preview → commit (same command system as 2D). Elevation unchanged during horizontal moves.

Materials — MeshStandardMaterial. Initial ranges: roughness 0.4–0.8, metalness 0.0–0.1. Ambient ~0.6, directional ~0.8 at [5,10,5].

Saved viewpoints — Selecting a named viewpoint smoothly transitions the camera to the stored position, target and projection.

9. Persistence & History

- Domain commands only. One drag = one command = one history entry. Multi-object operations are single composite commands.
- Commands are immutable history records and contain no renderer-specific or transient interaction state.
- **Every command carries mandatory inverse data** (e.g. `MoveFurniture.from`, `DeleteFurniture.snapshot`). Undo = apply inverse command; redo = re-apply original. `apply` never throws on replay of a stored command (Rule 19, 20).
- `apply(command, state)` is pure; IDs are supplied inside the command.

- History stores the commands (or a serialisable form of them).
 - Serialization is cloud-ready (stable project ID, schemaVersion, revision, serializable schema). Prototype may use local storage.
 - View/camera state and saved viewpoints are not part of design history.
-

10. Import / Export

Export (P0) — PDF floor plan (geometry, dimensions, furniture, labels, room area, schedule), 3D image, furniture schedule, materials/finishes list. Schedule columns: Tag | Item | Dimensions | Finish | Vendor / SKU (when present). Consume domain model, not UI screenshots.

Import — Bounded DXF subset is Deferred. Architecture leaves a clean import adapter boundary.

11. Phase / Priority Scope

P0 — Core spatial experience — Room geometry & editing with per-segment wall thickness, fixtures, furniture library + ghost, selection, move/resize/rotate, snap, collision, containment, constraints, live dimensions, Measure tool + room area display, inspector (including metadata), 2D (with dashed high-object representation), 3D visualization + basic manipulation, basic multi-selection (composite commands), clearance visualisation toggle, saved camera viewpoints, undo/redo, save/load, materials, PDF/image export.

P1 — Professional usability — Measurements polish, richer measurement workflows, persistent annotations, align / distribute, advanced grouping, assemblies UI, layers, library search/favourites, Focus Mode, room navigation polish, templates, full WALL / CEILING anchor support, imperial display.

P2 — Professional production — Versions, alternatives, full schedules, 3D export polish, bounded DXF, GLTF catalogue path, advanced materials.

P3 — Platform — Cloud, collaboration, sharing, catalogue services.

Early validation gate (before finishing all P0) — Create rectangle with unequal wall thicknesses → add door/window → place sofa + coffee table → toggle clearances → place high object (confirm dashed 2D) → switch 2D ↔ 3D and restore a saved viewpoint → inspect dimensions and metadata → export PDF with schedule. Plus the adoption question: *"would you use this instead of your current tool for one real project?"* If this loop fails with real designers, stop and fix it.

12. Testing, Performance & Visual Regression

Geometry unit tests — CCW, concave containment, self-intersection rejection, rotated footprints, centre-pivot & edge-anchored resize, SAT, vertical intervals, door/window constraints, snapping, coordinate round-trips, 1 mm quantization, rotation quantization, boundary edge cases, tolerance-aware assertions, per-segment wall thickness expansion, multi-object commands, inverse-application undo/redo, replay determinism, fixture vertical validity, clearance zone clipping.

Integration — Zustand ↔ domain commands, history, serialization, 2D/3D ↔ domain, selection sync, view switching, fixture attachment, saved viewpoint restore.

E2E core workflows — Create & furnish, constraint correction, room modification, 2D/3D round-trip, history, clearance visualisation, metadata in export.

Performance budgets

- Smooth drag ~60 FPS on representative desktop with 30 / 50 / 100 objects.
- No React domain-store updates or unnecessary geometry rebuilds on pointer move.
- Measure app load, project load, scene construction, first interactive frame, memory under repeated 2D/3D switching.
- Constraint evaluation may run every frame during drag (brute-force $O(n^2)$ acceptable for ≤ 100 objects in Phase 1; broad-phase AABB grid may be introduced later without changing the public validation API).

Visual regression — Room (including variable thickness), selection, invalid placement, dimensions, ghost, 3D scene, wall openings, dashed high objects, clearance overlays, inspector states.

13. Acceptance Criteria (Core Loop)

A professional designer can, without understanding the technology:

1. Create and shape a room with unequal wall thicknesses.
2. Add doors and windows.
3. Browse and ghost-place furniture.
4. Move / rotate / resize with live dimensional and constraint feedback.
5. Toggle clearance visualisation.
6. Recover naturally from invalid placements.
7. Inspect exact dimensions and metadata.
8. See high objects rendered as dashed in 2D.
9. Switch fluidly between precise 2D and exploratory 3D, restoring saved viewpoints.
10. Undo/redo confidently.
11. Save, reload, and produce a PDF / image output that includes a usable specification table.

The experience must feel like one coherent spatial-design application.

14. Future Architecture Boundaries

The domain and geometry layers already accommodate (without requiring immediate UI): Multiple rooms and room navigator · Vertical envelope (floor / wall / ceiling) · Assemblies · Layers · Hard/soft clearance rules · Full anchorType system (FLOOR | WALL | CEILING | SURFACE) · GLTF asset provider · Named versions & alternatives · Cloud persistence with revision/conflict detection · Collaboration (later).

Multi-room shared walls (named boundary): adjacent-room topology (walls shared between two rooms) is *not* representable in the Phase 1 single-room `EditWall` contract. When multi-room becomes P0, a shared-wall edit must be expressed as one cross-room CompositeCommand (both rooms' geometry + fixtures atomically), extending — not breaking — command atomicity. The vertex-ID scheme is chosen partly to keep this future extension clean.

These remain Foundations or Deferred until the P0 core loop is proven excellent.

Appendix A: Edge-Case & Implementation Defaults

These defaults govern edge cases across the domain engine, interaction model, and renderers during PO development. They are implementation contracts, not suggestions.

A1. Wall Geometry & Thickness

- The CCW room polygon defines the **interior finished face**.
- Each wall segment stores its own thickness (default 150 mm, configurable per segment). Thickness expands **outward** only. Interior usable area is never reduced by thickness.
- Wall centreline is derived visual/reference geometry only; the domain never stores or reasons about it.
- Fixture attachment coordinate (`offsetAlongWall`) is measured along the interior wall boundary.
- Fixture orientation is derived from the boundary segment.
- Fixture opening and constraint geometry are calculated from the interior face.
- Wall thickness is a rendering/attachment concern; it does not alter the domain polygon.
- **Vertex-ID rule (replaces index remapping)**: room vertices carry stable, immutable string IDs. Wall segments reference `startVertexId` / `endVertexId` — never array indices. Vertex insertion/deletion therefore cannot invalidate other segments' references, and EditWall commands/inverses restore geometry by ID. The interaction layer never remaps indices.

A2. Conflict Resolution (Snap vs. Constraint)

- Hard spatial constraints and collision rules always override snapping.
- If a candidate snap position would produce a hard violation, the snap is discarded.
- The object is left at the nearest valid position (or the last valid committed position if no nearer valid point exists within the current drag).
- A constraint warning is shown immediately.

A3. Room Editing & Existing Objects

- Moving or reshaping a wall never automatically translates interior furniture or fixtures.
- **Pipeline exception (binding)**: EditWall commands validate the *room polygon itself* (CCW, simple, error codes) but do **not** re-validate furniture/fixture constraints as a blocking condition. A wall edit is committed even if furniture becomes invalid.
- After commit, every furniture instance and fixture is re-validated. Any object now violating a hard constraint is shown invalid in the UI (derived) and remains committed until the user resolves it. Pre-commit impact preview behaviour is defined in Appendix B, B1.
- Soft constraint violations are indicated visually but do not block the wall edit itself.
- Undo of the wall edit applies the inverse command; validity is re-derived and therefore also restored.
- **No-deadlock rule (binding)**: objects made invalid by a wall edit are never stuck. Subsequent moves/resizes/rotates of an already-invalid object are evaluated against the *delta* of its violations (commit if no new violation and no severity increase) — defined in Phase 1 §6 escape rule. The B5 "Fix position" affordance is the one-click exit to the nearest fully valid position.

A4. Fixture Placement (Ghost)

- While dragging a door or window from the library the fixture is an unattached ghost that follows the pointer.
- `fixtureWallSnapDistance` is a configurable interaction preference (default 0.5 m).

- When the pointer comes within the snap distance of a wall segment the ghost: snaps onto the wall; orients to the wall direction; switches to wall-offset positioning; clamps within the wall segment bounds (margin at each end equal to half the fixture width or 50 mm, whichever is larger).
- Escape cancels; click/tap commits only if the final attachment is valid.
- Domain validation additionally rejects any committed fixture whose interval falls outside its wall segment (with the defined end margin) or outside the wall's vertical extent (§4 fixture vertical validity).

A5. Invalid Inspector Inputs

- Every numeric field in the Inspector performs live domain validation on change.
- Hard-constraint violation → field outline turns red + tooltip shows the primary violation.
- On Blur or Enter: if still invalid → value reverts to the last committed valid number; if valid → value is quantized and committed as a single domain command.
- Soft violations are shown but do not prevent commit.

A6. Duplication Near Boundaries

- Default duplicate offset = **+0.1 m** on both X and Y (one default grid unit).
- A new persistent instance ID is generated *outside* **apply** and carried inside the command; the instance is **not** added to domain state until a valid commit occurs.
- If the offset position is hard-invalid, the duplicate is not committed; the ghost remains attached to the pointer so the user can place it in a valid location.
- Escape discards the ghost; no history entry is created until a valid placement occurs.

A7. 3D Camera & Wall Visibility

- Walls between the camera eye and the room centre are automatically faded (or temporarily lowered) while orbiting. Pure rendering concern.
- The behaviour is disabled in orthographic top-down views and when the camera is looking nearly straight down.

A8. Vertical Stacking & Elevation

- All furniture defaults to **elevation = 0.0 m** (floor) on placement.
- Categories tagged **decor**, **lighting**, or **tabletop** may surface-snap (Architectural Foundation; intentionally out of Phase 1).
- Vertical interval collision remains active.
- Manual elevation editing is always available in the Inspector.

A9. Multi-Selection & Mixed Values

- When multiple objects are selected, any property that differs shows "Mixed".
- Editing a Mixed numeric field applies the new value to all properties in the **defined-semantics whitelist** (Appendix B, B4). Others display as unavailable.
- If any object would become hard-invalid, the entire edit is rejected, the field reverts, and a single aggregated warning is shown.
- Soft violations are applied and indicated per-object.
- Group operations are emitted as a single CompositeCommand.

A10. Quantization & Tolerance Interaction

- During pointer movement all calculations remain full floating-point.
- On commit: resolve snap → quantize (linear → 1 mm, rotation → 0.1° or active angular snap) → re-evaluate hard constraints on quantized values → if valid, emit one domain command; if invalid, reject and retain the last valid state.
- **Commands arriving at `apply` must already be quantized.** `apply` may assert this (debug) but never repairs or re-quantizes.
- Geometric predicates use `EPSILON = 1e-6 m`. A footprint within EPSILON of the boundary is touching (valid unless clearance forbids it).
- Explicit edge-case: after quantization, a corner that lands exactly on the boundary is touching (valid). Quantization must never push a previously valid touching point 1e-6 outside.

A11. History & Transient State

- Only successful commits create history entries.
- Ghost placement, invalid drags, cancelled operations, and view/camera changes never enter history.
- **Undo = inverse-application of the stored command** (Rule 20). Redo = re-application. Replay of stored commands must never throw.
- Validity is re-derived after each undo/redo step.

A12. Performance Guardrails (Pointer Path)

- Pointer movement may update local renderer transforms, previews, snap indicators, dimensions, and constraint overlays.
- Persistent domain state, React-wide rendering, and expensive geometry reconstruction occur only when required by the interaction or on commit.
- Snap and constraint evaluation may run every frame; results are cached per object for the duration of the drag.
- Brute-force $O(n^2)$ validation is acceptable for ≤ 100 objects in Phase 1. Broad-phase may be added later without changing the public validation API.

A13. Cut-Plane & Dashed Representation

- Default cut-plane height = 1.2 m (configurable).
- Any FurnitureInstance where `elevation + height > cutPlaneHeight` is rendered with dashed lines in the 2D view.
- Pure 2D rendering convention; does not affect collision, constraints, or 3D.

A14. Clearance Visualisation

- A user toggle shows the hard and soft clearance zones defined by the selected object's clearancePolicies.
- Zones are rendered as dashed or lightly hatched overlays in both 2D and 3D.
- Visualisation is non-interactive and does not affect commit behaviour.

A15. Saved Viewpoints

- Saved viewpoints store camera position, target, projection type, and optional room association.
- Restoring a viewpoint performs a smooth camera transition.
- Viewpoints are independent of design history.

A16. Locked Objects

- Locked objects still participate in collision and containment checks.
- Move, resize, rotate and delete commands targeting a locked object are rejected (in the commit pipeline, before command emission).
- Locked objects remain selectable and visible in the inspector.

These defaults are binding for P0. Any deviation requires an explicit update to this Appendix. UX interaction defaults are binding via **Appendix B** (separate document).

End of Specification V2 Rev 3 (Frozen)